

7) define NUM-PROCESSES

13th Nov 2020

20233102

131

classmate

Date
Page

```
int available [NUM_RESOURCES] = {2, 1, 0, 0};
int max [NUM_PROCESSES] [NUM_RESOURCES] = {{4, 3, 3, 1}, {3, 2, 1, 2}, {2, 1, 1, 2}, {7, 5, 3, 1}, {1, 1, 2, 1}};
int need [NUM_PROCESSES] [NUM_RESOURCES];
pthread_mutex_t lock;
void calculate_need();
for (int i = 0; i < NUM_PROCESSES; i++) {
    for (int j = 0; j < NUM_RESOURCES; j++) {
        need[i][j] = max[i][j] - allocation[i][j];
    }
}
```

int is_safe();

int work [NUM_RESOURCES];

int finish [NUM_PROCESSES] = {0};

for (int i = 0; i < NUM_RESOURCES; i++) {

work[i] = available[i];

int Count = 0;

while (Count < NUM_PROCESSES) {

int found = 0;

for (int i = 0; i < NUM_PROCESSES; i++) {

if (finish[i] == 0) {

int can_allocate = 1;

for (int j = 0; j < NUM_RESOURCES; j++) {

if (need[i][j] > work[j]) {

can_allocate = 0;

break;

if (can_allocate) {

for (int j = 0; j < NUM_RESOURCES; j++) {

work[j] += allocation[i][j];

finish[i] = 1;

classmate

Date
Page

finish[i] = 1;

Count++;

found = 1;

if (found == 0) {

return 0;

return 1;

int request_resources (int process_id, int request[])

pthread_mutex_lock (&lock);

for (int i = 0; i < NUM_RESOURCES; i++) {

if (request[i] > need[process_id][i]) {

pthread_mutex_unlock (&lock);

printf ("P%d request exceeds its need request", process_id);

return 0;

return 1;

for (int i = 0; i < NUM_RESOURCES; i++) {

if (request[i] > available[i]) {

pthread_mutex_unlock (&lock);

printf ("P%d request exceeds available resources", process_id);

return 0;

Request denied. In process_id;

return 0;

for (int i = 0; i < NUM_RESOURCES; i++) {

available[i] -= request[i];

allocation[process_id][i] += request[i];

need[process_id][i] = request[i];

return 1;


```
if (r == safe) {
```

```
for (int i = 0; i < NUM_RESOURCES; i++) {
```

```
    available[i] += request[i];
```

```
    allocation[process-id][i] -= request[i];
```

```
}
```

```
pthread_mutex_unlock(&lock);
```

```
printf("P%d request leads to unsafe state\n", process-id);
```

```
Request denied in process-id;
```

```
return 0;
```

```
printf("P%d request granted in process-id\n", process-id);
```

```
pthread_mutex_unlock(&lock);
```

```
return 1;
```

```
}
```

```
void release_resources(int process-id) {
```

```
    pthread_mutex_lock(&lock);
```

```
    for (int i = 0; i < NUM_RESOURCES; i++) {
```

```
        available[i] += allocation[process-id][i];
```

```
        allocation[process-id][i] = 0;
```

```
        need[process-id][i] = max[process-id][i];
```

```
}
```

```
printf("P%d releasing resources in process-id\n", process-id);
```

```
pthread_mutex_unlock(&lock);
```

```
}
```

```
void *process_thread(void *args) {
```

```
    int process-id = *(int *)args;
```

```
    int request[NUM_RESOURCES];
```

```
    for (int i = 0; i < NUM_RESOURCES; i++) {
```

```
        request[i] = rand() % (need[process-id][i] + 1);
```

```
}
```

```
printf("P%d requesting: P%d, %d, %d\n", process-id,
```

```
request[0], request[1], request[2]);
```

```
request = (request - resources (process-id, request));
```

```
release_resources(process-id);
```

```
}
```

```
pthread_exit(NULL);
```

```
} pthread
```

```
int main() {
```

```
    pthread_t threads[NUM_PROCESSES];
```

```
    int process-id (NUM_PROCESSES);
```

```
    pthread_mutex_init(&lock, NULL);
```

```
    calculate_need();
```

```
    for (int i = 0; i < NUM_PROCESSES; i++) {
```

```
        process-id[i];
```

```
        pthread_create(&threads[i], NULL, process_thread,
```

```
        &process-id[i]);
```

```
    } for (int i = 0; i < NUM_PROCESSES; i++) {
```

```
        pthread_join(threads[i], NULL);
```

```
    } printf("All processes completed successfully\n");
```

```
    pthread_mutex_destroy(&lock);
```

```
    return 0;
```

```
}
```